Type Hinting

Type hinting is a way to specify the type of a variable in PHP. This is useful for both readability and maintainability of your code. It also helps to catch errors early on in the development process. In Laravel, this is completely optional, but it is a good practice to use it. We are using it in our arguments. We can also use it with return types for our functions in the controller. Laravel has custom types that we can use. Here are some common ones:

- `Illuminate\Http\Request` - This is the request object that is passed to the controller method. It contains information about the HTTP request.
- `Illuminate\Http\Response` - This is the standard HTTP response object returned from a controller method. It can represent any kind of HTTP response.
- `Illuminate\Http\RedirectResponse` - This response type is used to redirect the user to another URL, often after a form submission.
- `Illuminate\Http\JsonResponse` - This is a response type specifically for returning JSON data, commonly used in APIs.
- `Illuminate\View\View` - This is the view object returned when rendering a Blade template, typically used in web applications.
- `Illuminate\Support\Collection` - This is a collection object returned from a controller method, often used to handle groups of models or data sets.
- `Illuminate\Auth\Access\Response` - This response type is used in authorization checks, allowing you to provide feedback on authorization decisions.
- `Illuminate\Pagination\LengthAwarePaginator` - This is a paginator object returned when paginating a collection of results, often used in index methods.
- `Symfony\Component\HttpFoundation\StreamedResponse` - This is used for streaming content, such as when generating large files for download.
- `Symfony\Component\HttpFoundation\BinaryFileResponse` - This is used for sending files to the user, typically used for file downloads.
- `Illuminate\Contracts\Routing\ResponseFactory` - This contract allows for creating various types of responses, like JSON, view, or download responses.

There are others for insance, the Eloquent ORM has it's own types.

Let's add return types to our `JobController` methods. Right now, a bunch of them are just returning strings. So those will be changed later.

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\View\View; // Add this line

class JobController extends Controller
{
    // @desc   Show all jobs
    // @route  GET /jobs
    public function index(): View
    {
```

```
        $title = 'Available Jobs';
        $jobs = [
            'Software Engineer',
            'Web Developer',
            'Data Scientist',
        ];

        return view('jobs/index', compact('title', 'jobs'));
    }

    // @desc   Show create job form
    // @route  GET /jobs/create
    public function create(): View
    {
        return view('jobs.create');
    }

    // @desc   Store a new job
    // @route  POST /jobs
    public function store(Request $request): string
    {
        $title = $request->input('title');
        $description = $request->input('description');

        return "Title: $title, Description: $description";
    }

    // @desc   Show a single job
    // @route  GET /jobs/{id}
    public function show(string $id): string
    {
        return "Showing job $id";
    }

    // @desc   Show the form for editing a job
    // @route  GET /jobs/{id}/edit
    public function edit(string $id): string
    {
         return "Edit job $id";
    }

    // @desc   Update a job
    // @route  PUT /jobs/{id}
    public function update(Request $request, string $id): string
    {
         return "Update job $id";
    }

    // @desc  Delete a job
    // @route DELETE /jobs/{id}
    public function destroy(string $id): string
    {
          return "Delete job $id";
```

```
    }
}
```

As you can see to use `View` we need to use `use Illuminate\View\View;` at the top of the file.

In the HomeController, let's add the `View` type hinting to the `index` method.

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Job;
use Illuminate\View\View;

class HomeController extends Controller
{
    public function index(): View
    {
        return view('pages.home');
    }
}
```

Again, this is optional but it does help with readability and maintainability of your code. I will be using it in the rest of the course.